Relazione - Versione 10: Decoder-Only Transformer

Introduzione

La Versione 10 trasforma il singolo Transformer block esplicito della Versione 9 in un piccolo language model Transformer decoder-only costruito con componenti riutilizzabili.

Restano invariati il corpus inglese di 440 caratteri, il vocabolario di 27 caratteri, il contesto di quattro caratteri, gli embedding di carattere e posizionali a 8 dimensioni, l'obiettivo causale di predizione del carattere successivo, lo split training/validation riproducibile, l'ottimizzazione mini-batch e la generazione autoregressiva. Il cambiamento principale è architetturale e organizzativo: un `TransformerBlock` riutilizzabile viene istanziato due volte, i blocchi usano parametri indipendenti e l'output del primo blocco diventa l'input del secondo.

```
Caratteri
   |
Identificatori interi
   |
Contesto di 4 caratteri + 4 target sequenziali
   |
Embedding carattere + posizione
   |
Transformer block 1
   |
Transformer block 2
   |
Logits del vocabolario a ogni posizione
   |
Testo generato
```

Obiettivo della Versione 10

- preservare corpus, vocabolario, contesto di quattro caratteri e obiettivo di training della Versione 9;
- raggruppare attention, connessioni residuali, LayerNorm e operazioni feed-forward in un Transformer block riutilizzabile;
- impilare due Transformer block indipendenti preservando il flusso causale dell'informazione;
- raggruppare i parametri del modello in modo che il training non debba più passare manualmente ogni variabile Transformer;
- mantenere esplicita la matematica rendendo al tempo stesso l'implementazione più facile da estendere;
- verificare indipendenza dei blocchi, forme, numero di parametri, causal masking, gradienti, aggiornamenti, riproducibilità e generazione.

Architettura

Il flusso completo della Versione 10 è:

```
Corpus: 440 caratteri
   |
Vocabolario: 27 caratteri
   |
436 esempi sequenziali contesto-target
   |
Tensore input:  (436, 4)
Tensore target: (436, 4)
   |
Embedding carattere: (27, 8) + posizioni: (4, 8)
   |
Transformer block 1: (4, 8) -> (4, 8)
   |
Transformer block 2: (4, 8) -> (4, 8)
   |
Logits del vocabolario: (4, 27) per esempio
   |
Split 80 / 20 riproducibile + aggiornamenti mini-batch
   |
Generatore autoregressivo a finestra scorrevole
```

Transformer block riutilizzabili

Ogni Transformer block riceve una rappresentazione della sequenza con forma `(batch, 4, 8)`. All'interno del blocco, query, key e value vengono proiettati a 16 dimensioni, la causal scaled dot-product attention combina solo la posizione corrente e quelle precedenti e una proiezione di output riporta il risultato dell'attention a 8 dimensioni. Connessioni residuali, LayerNorm e una rete feed-forward position-wise `8 -> 16 -> 8` preservano la forma della rappresentazione.

I due blocchi hanno la stessa struttura ma variabili addestrabili indipendenti. Il secondo blocco elabora quindi la rappresentazione già trasformata dal primo blocco invece di riutilizzare gli stessi parametri.

Finestre di contesto, split e mini-batch

```
esempi training   = 348
esempi validation = 88
batch size         = 32
aggiornamenti per epoca = 11
```

Solo gli esempi di training vengono mescolati e usati per gli aggiornamenti. La validation misura la loss senza modificare i parametri. Le finestre vicine si sovrappongono, quindi questo split resta un indicatore didattico leggero e non una stima robusta della generalizzazione su testo non visto.

Parametri addestrabili

Il modello contiene 23 variabili TensorFlow addestrabili e precisamente 2064 parametri addestrabili:

```
Embedding carattere        27 x 8  = 216
Embedding posizionali       4 x 8  =  32
Transformer block 1                  = 800
Transformer block 2                  = 800
Pesi output                8 x 27 = 216
----------------------------------------
Totale                               = 2064
```

Ogni Transformer block contribuisce esattamente 800 parametri:

```
Proiezione query             8 x 16 = 128
Proiezione key               8 x 16 = 128
Proiezione value             8 x 16 = 128
Proiezione output attention 16 x 8  = 128
LayerNorm 1 scala + shift    8 + 8   =  16
Input feed-forward           8 x 16 = 128
Output feed-forward         16 x 8  = 128
LayerNorm 2 scala + shift    8 + 8   =  16
----------------------------------------
Per blocco                           = 800
```

Dettagli del Transformer block

Per l'input X, ogni blocco calcola le proprie proiezioni query, key e value. La causal mask viene applicata prima del softmax e i valori contestualizzati risultanti vengono proiettati nuovamente nella rappresentazione del modello a 8 dimensioni:

```
Q = X Wq
K = X Wk
V = X Wv

scores = Q K^T / sqrt(dk)
masked_scores = causal_mask(scores)
A = softmax(masked_scores)
H = A V
```

Non vengono usati termini di bias nelle proiezioni di attention, feed-forward o vocabolario. Entrambe le operazioni LayerNorm usano parametri addestrabili di scala e shift.

Causal mask

```
[[1, 0, 0, 0],
 [1, 1, 0, 0],
 [1, 1, 1, 0],
 [1, 1, 1, 1]]
```

Loss, training e riproducibilità

La sparse softmax cross-entropy viene calcolata in tutte e quattro le posizioni della sequenza. Le loss per posizione hanno forma `(batch, 4)` e la loro media diventa l'obiettivo scalare. `tf.GradientTape` calcola i gradienti per tutte le 23 variabili addestrabili. I lookup degli embedding possono produrre gradienti sparsi `IndexedSlices`; poiché il modello è molto piccolo, questi gradienti vengono convertiti in tensori densi prima dell'aggiornamento manuale con gradient descent mini-batch.

```
Learning rate: 1.0
Batch size:    32
Epoche:        100
Seed:          42
```

Risultato del training salvato

```
Loss training iniziale:     3.295300
Loss training finale:       1.844497
Loss validation iniziale:   3.291214
Loss validation finale:     2.177593
Migliore loss validation:   2.126416
Migliore epoca validation:  98
```

La loss di training diminuisce nettamente. Anche la validation migliora e raggiunge il valore migliore verso la fine del training. In questa esecuzione il modello più profondo ottiene loss salvate leggermente inferiori rispetto alla Versione 9, ma il dataset minuscolo con finestre sovrapposte e l'uso di un solo seed non consentono un confronto forte sulla generalizzazione. Tra ambienti TensorFlow possono comparire piccole differenze numeriche.

Probabilità apprese

Per il contesto `mode`, le probabilità finali più alte nell'esecuzione salvata sono:

```
spazio  0.3901
t       0.1992
l       0.1369
r       0.0717
c       0.0697
s       0.0373
```

Ispezione della causal attention impilata

```
Block 1
       m       o       d       e
m  1.0000  0.0000  0.0000  0.0000
o  0.4487  0.5513  0.0000  0.0000
d  0.3645  0.2860  0.3495  0.0000
e  0.3506  0.1381  0.3166  0.1948

Block 2
       m       o       d       e
m  1.0000  0.0000  0.0000  0.0000
o  1.0000  0.0000  0.0000  0.0000
d  0.0410  0.0748  0.8842  0.0000
e  0.9923  0.0000  0.0077  0.0000
```

Ogni riga somma a circa 1 e la massima attention verso posizioni future è esattamente 0.0 in entrambi i blocchi, confermando che lo stacking preserva il vincolo causale.

Generazione autoregressiva

La generazione usa una finestra scorrevole di quattro caratteri. A ogni passo il contesto completo attraversa entrambi i Transformer block. Solo i logits dell'ultima posizione della sequenza vengono convertiti in probabilità e campionati; poi il carattere più vecchio del contesto viene eliminato e il nuovo carattere viene aggiunto.

Con seed 42, cinque campioni indipendenti dal contesto `mode` nell'esecuzione salvata sono:

```
s, c, t, r, spazio
```

La dimostrazione da 300 caratteri inizia così:

```
modes spaurs matsin.
atindprs macorte macod nspesinct.
asiond caned ngom inet.
eromel s adve math inan tunctong comake spredin m ne coms le tin...
```

Controlli inclusi

- i tensori input e target hanno entrambi forma `(436, 4)`;
- il modello contiene esattamente due Transformer block con variabili addestrabili indipendenti;
- tutte le 23 variabili addestrabili hanno le forme previste e totalizzano esattamente 2064 parametri;
- entrambi i blocchi preservano la rappresentazione `(batch, 4, 8)` e producono matrici di attention `4 x 4`;
- i pesi di attention verso il futuro sono esattamente zero in entrambi i blocchi e ogni riga di attention somma a 1;
- le distribuzioni di probabilità del carattere successivo sono finite e sommano a 1;
- tutti i gradienti sono presenti e finiti e ogni variabile addestrabile cambia durante il training;
- gli indici training e validation sono separati e le cronologie delle loss registrate restano finite;
- retraining e generazione autoregressiva sono riproducibili con lo stesso seed.

La sezione di test del notebook termina con `All checks passed.`

Cosa insegna la Versione 10

La Versione 10 è il primo passaggio del progetto in cui l'architettura Transformer viene organizzata come stack decoder-only riutilizzabile anziché come singolo blocco esplicito. La progressione concettuale principale è:

```
contesto -> embedding + posizioni
         -> Transformer block 1
         -> Transformer block 2
         -> logits del vocabolario a ogni posizione
         -> loss su tutte le posizioni
         -> campionamento autoregressivo dall'ultima posizione
```

La differenza importante rispetto alla Versione 9 non è una nuova equazione di attention. La matematica interna a ogni blocco viene preservata, mentre cambiano la profondità del modello e la struttura del software: i parametri vengono raggruppati in componenti riutilizzabili e blocchi indipendenti possono essere impilati in sequenza.

Limitazioni

- la lunghezza del contesto resta fissa a quattro caratteri e il corpus contiene solo 440 caratteri;
- il modello resta character-level ed è intenzionalmente molto più piccolo di un large language model;
- ogni Transformer block usa un solo meccanismo di attention invece della multi-head attention;
- vengono usati solo due Transformer block; non sono presenti dropout o learning-rate schedule;
- le finestre sovrapposte compaiono nello split casuale training/validation, quindi la validation non è un benchmark robusto di generalizzazione;
- la generazione mantiene i parametri dell'epoca finale invece di ripristinare automaticamente il miglior checkpoint di validation;
- l'implementazione resta volutamente esplicita e non usa ancora un'astrazione Keras di livello superiore.

Conclusione

La Versione 10 trasforma con successo il singolo Transformer block causale della Versione 9 in un piccolo Transformer decoder-only composto da due blocchi riutilizzabili e indipendenti. Mantiene l'ambiente character-level piccolo e ispezionabile, introducendo al tempo stesso l'organizzazione del modello e la profondità necessarie per la fase successiva del progetto.

Il modello resta intenzionalmente minuscolo, ma la struttura decoder-only fondamentale è ora presente. Una versione successiva potrà passare dai token a carattere alla tokenizzazione subword e a dataset documentati più grandi, preservando l'architettura sviluppata qui.